

김만중 | Software Developer

새로운 기술에 대한 호기심을 성장의 동력으로 삼습니다.

변화하는 기술 트렌드 앞에서도 두려움 없이 배우고 적용하는 것을 주저하지 않습니다.
이러한 태도로 멀티플레이 환경 + 강화학습 AI 분야를 선택하여 게임 프로젝트를 완수했습니다.
그 결과, 교내 캡스톤디자인 장려상, 대전 게임 브릿지 데이 우수상을 수상했습니다.

끊임없이 질문하고 탐구하며, 더 나은 코드를 고민합니다.

[게임잼 프로젝트](#)에서 확장성만을 고려한 상속 구조가 오히려 팀원들의 작업 효율을 떨어뜨리는 경험을 했습니다.
이 경험을 통해 동료가 쉽게 이해하고 확장할 수 있는 친절한 코드를 작성하기 위해 노력을 기울이고 있습니다.

목차

[Unity] The Zoo - 멀티플레이 게임 개발 프로젝트.....	2
[Unity] ML-Agents 기반 자율주행 강화학습 실험	12
[Unity] Throw it – 2025 인디 게임어스 게임잼	19
[Unity] WALL – 2025 게임마당 게임잼.....	23
[Web] Hanbat Box - AWS S3 기반 파일 공유 서비스.....	26
[Web] Dyslexia - 생성형 AI 기반 난독증 아동 교재 생성 서비스.....	36
[Web] UCSI University Internship - 강의실 배정 최적화 시스템	42

[Unity] The Zoo - 멀티플레이 게임 개발 프로젝트

항목	내용
진행 기간	2025.03 ~ 2025.12 (10개월)
팀 구성	2명 (팀장)
사용 기술	Unity, ML-Agents, Netcode for GameObjects
핵심 목표	인간을 모방하는 AI와 함께하는 심리전 중심의 멀티플레이 게임 구현
수상 실적	 inD 게임 브릿지 데이 우수상  교내 캡스톤디자인 장려상

기획 배경

- 단순한 패턴의 NPC가 아닌 강화학습을 통해 인간처럼 행동하여 술래를 속이는 에이전트를 개발하여 마피아 류의 심리전 구현
- 한정된 공간에서 끊임 없는 추격전을 위해 행성 형태의 구형 맵과 중력 시스템 도입
- 분산 권한 토폴로지를 채택하여 각 클라이언트에게 쾌적한 조작감을 제공하고 지연 없는 멀티플레이 환경 구축

주요 역할

- 시스템 설계: 구형 중력 물리 시스템 및 캐릭터 제어 로직 구현
- 네트워크 구축: UGS 기반의 인증 로비 릴레이 연결 및 전체 게임 세션 라이프사이클 구현
- AI 개발: 플레이어 행동을 모방하고 의심도 기반으로 생존 전략을 학습할 강화학습 AI 설계
- 게임 로직: 역할 시스템, 미션 수행 이벤트 스케줄링 등 전체 게임 규칙 구현

프로젝트 성과 및 회고

성과

- 지능형 AI 구현: 시각 정보와 의심도 수치를 결합한 보상 설계로 인간 플레이어와 구분이 어려운 수준의 자연스러운 회피 기동 학습 성공
- 안정적인 네트워크: 네트워크 연결 흐름을 구축하고 호스트 권한 기반의 이벤트 스케줄러로 동기화 오차 최소화
- 물리 시스템 최적화: 구형 행성 표면에서의 이동과 낙백 처리를 위한 커스텀 중력 및 물리 연산 시스템 구축

회고 및 개선점

- 상속보다 컴포지션: 상속 구조의 한계를 느끼고 기능을 컴포넌트로 분리하여 유지보수성을 높임
- AI 추론 부하 분산: 호스트가 AI 연산을 전담하여 발생하는 성능 편차 문제를 확인하고 추후 소유권 이전을 통한 연산 분산의 필요성을 확인함
- 보상 설계 노하우: 단순 행동 보상보다 의심도라는 지표를 도입하여 AI가 안전 구간 내에서 행동하도록 유도하는 것이 학습 효율에 유리함을 확인함

핵심 구현 상세

1. 강화학습 AI 에이전트 (HiderTrainAgent.cs)

ML-Agents 기반으로 숨는 역할을 수행하며 시각 정보와 내부 상태를 분석해 생존 전략을 학습함

1. 다중 센서 관측 및 행동 결정

- Ray 센서와 벡터 관측을 결합하여 주변 환경을 인식하고 술래가 자신을 보고 있는지를 수치화하여 관측에 포함함

```
public override void CollectObservations(VectorSensor sensor)
{
    // 1. 자신의 벡터 정보 관측 (위, 앞, 오른쪽)
    sensor.AddObservation(transform.up.normalized);
    sensor.AddObservation(transform.forward.normalized);
    sensor.AddObservation(transform.right.normalized);

    // 2. 현재 상태 정보 관측
    sensor.AddObservation(canMove); // 이동 가능 여부
    sensor.AddObservation(isGround); // 착지 여부

    // 3. 술래와의 관계 관측 (시야각 내적)
    // 술래의 전방 벡터와 나를 향한 벡터의 내적을 계산하여
    // 술래가 나를 보고 있는지(-1 ~ 1)를 수치화
    sensor.AddObservation(GetSeekerViewDot());

    // 4. 행동 리스크 관측 (의심도)
    sensor.AddObservation(suspicion);
}
```

2. 이산적 행동 결정

- 신경망의 출력값을 해석하여 이동, 점프, 스핀 등의 행동을 수행함

```
public override void OnActionReceived(ActionBuffers actions)
{
    var branch = actions.DiscreteActions;

    // Branch 0: 이동 방향 결정 (정지, 전, 후, 좌, 우)
    var moveDir = Vector2.zero;
    switch (branch[0]) { ... }

    // Branch 1: 점프 실행 여부
    if (branch[1] == 1) Jump();

    // Branch 2: 스핀(대시) 실행 여부
    if (branch[2] == 1) Spin();
}
```

3. 의심도 기반 보상 설계

- 단순 생존뿐만 아니라 자연스러운 행동을 유도하기 위해 의심도 시스템을 도입하여 보상 함수에 적용함

```
private void HandleRewards()
{
    // 1. 자연스러운 이동 유도
    // 이동 중일 때 시선이 이동 방향과 일치하면 보상 부여
    if (rBody.linearVelocity.magnitude > 0.005f)
    {
        var gravityDir = -planet.GetGravityDirection(transform.position).normalized;
        // 접선 방향 속도와 전방 벡터 비교
        var tangentVelocity = Vector3.ProjectOnPlane(rBody.linearVelocity.normalized,
gravityDir).normalized;
        var tangentForward = Vector3.ProjectOnPlane(transform.forward,
gravityDir).normalized;

        if (Vector3.Dot(tangentForward, tangentVelocity) > 0.95f)
            AddReward(0.0005f);
    }

    // 2. 술래 시야각에 따른 보상/벌점
    var dot = GetSeekerViewDot();
    if (dot >= 0f)
    {
        if (dot > 0.8f) AddReward(-0.001f); // 정면 노출 시 벌점
        else if (dot < 0.5f) AddReward(0.001f); // 시야 밖/측면 위치 시 보상
    }

    // 3. 의심도 관리 및 대기 보상
    HandleSuspicion();
    RewardIdle();
}

private void HandleSuspicion()
{
    // 의심도가 안전 구간 내에 있으면 보상, 벗어나면 패널티
    // 너무 완벽하게 숨거나 튀는 행동을 방지하여 인간 유사성 유도
    if (suspicion < suspicionSafeZone.x || suspicion > suspicionSafeZone.y)
    {
        AddReward(suspicionPenalty);
    }
    else
    {
        AddReward(suspicionBonus);
    }

    // 의심도 임계치 초과 시 게임 오버 (학습 종료)
    if (suspicion >= maxSuspicion)
    {
        AddReward(-1f);
        EndEpisode();
    }
}
```

2. 구형 행성 중력 (PlanetGravity.cs)

캐릭터들이 구형 행성 표면에 붙어 이동할 수 있도록 하는 커스텀 중력 시스템

```
// 행성 중심을 향하는 방향 벡터 계산
public Vector3 GetGravityDirection(Vector3 position)
{
    return (center - position).normalized;
}

// 등록된 모든 객체에 중력 적용
private void ApplyGravity()
{
    foreach (var rb in affectedBodies)
    {
        if (!rb) continue;
        // 행성 중심 방향으로 지속적인 가속도 적용
        rb.AddForce(GetGravityDirection(rb.position) * GravityStrength,
            ForceMode.Acceleration);
    }
}

// 구체 표면 좌표 반환 (스폰 위치 계산용)
public Vector3 GetSurfacePoint(out Vector3 normal)
{
    normal = Vector3.up;
    return center + Vector3.up * radius;
}
```

3. 네트워크 세션 관리 (ConnectionManager.cs)

복잡한 UGS 연결 과정을 빌더 패턴과 비동기 흐름 제어로 추상화하여 안정성을 확보함

1. 연결 흐름 추상화

- 접속 방식에 상관없이 동일한 예외 처리와 UI 갱신 흐름을 타도록 구조화함

```
private async Task HandleConnectionFlowAsync(Func<Task<ISession>> sessionFunc)
{
    try
    {
        OnSessionConnectStart(); // 1. 로딩 UI 표시 및 초기화

        CurrentSession = await sessionFunc.Invoke(); // 2. 주입된 연결 로직 실행

        // 연결 성공 후 후처리 로직
    }
    catch (Exception e)
    {
        // 3. 에러 핸들링 통합
        NotifyUser(GetReadableErrorMessage(e));
        OnSessionConnectFail();
        throw;
    }
}
```

2. 세션 옵션 빌더 및 예외 처리

```
// 빌더 패턴을 통한 세션 옵션 구성
private async Task CreateSessionAsync(string sessionName, ...)
{
    var options = new SessionOptionBuilder()
        .Name(sessionName)
        .SeekerCount(1)
        .IsPrivate(true)
        .BuildCreate();

    await HandleConnectionFlowAsync(async () =>
        await MultiplayerService.Instance.CreateSessionAsync(options));
}

// 에러 메시지 매핑
private string GetReadableErrorMessage(Exception e)
{
    if (e is LobbyServiceException le)
    {
        switch (le.Reason)
        {
            case LobbyExceptionReason.RateLimited: return "잠시 후 다시 시도해주세요";
            case LobbyExceptionReason.LobbyNotFound: return "방을 찾을 수 없습니다";
        }
    }
    return "연결 중 오류가 발생했습니다";
}
```

4. 게임 매니저 및 데이터 동기화 (GameManager.cs)

전역 상태 관리 및 네트워크 리스트를 통한 실시간 플레이어 정보 동기화

```
// 플레이어 리스트 변경 감지 및 동기화
private void OnPlayersChanged(NetworkListEvent<PlayerData> changeEvent)
{
    switch (changeEvent.Type)
    {
        case NetworkListEvent<PlayerData>.EventType.Add:
        case NetworkListEvent<PlayerData>.EventType.Value:
            // 접속한 플레이어 정보를 로컬 딕셔너리에 캐싱
            var pName = Util.GetPlayerNameWithoutHash(changeEvent.Value.name.Value);
            playerDict[changeEvent.Value.clientId] =
                (pName, changeEvent.Value.type, changeEvent.Value.role);
            break;
    }
}

// 게임 시작 제어 (호스트 권한)
internal void GameStart()
{
    try
    {
        if (!ConnectionManager.Instance.CurrentSession.IsHost) return;
        if (!CanGameStart()) throw new Exception("플레이어들이 준비되지 않았습니다");

        // 씬 전환 RPC 호출 및 세션 잠금
        LoadSceneRpc(Util.InGameSceneName);
        ConnectionManager.Instance.LockSessionAsync();
    }
    catch (Exception e)
    {
        InformationPopup.instance.ShowPopup(e.Message);
    }
}
```

5. 서버 권한 이벤트 스케줄러 (PlayManager.cs)

네트워크 지연과 상관없이 정확한 타이밍에 게임 이벤트를 발생시키기 위해 서버 시간을 기준 시계로 사용함

1. 게임 흐름 초기화

```
private void StartGameFlow()
{
    // 맵 생성 시드 공유
    if (IsSessionOwner)
        sharedSeed.Value = Random.Range(0, 1_000_000);

    // 서버 시작 시간 기록
    gameStartTimeServer = NetworkManager.ServerTime.Time;
    schedule.Clear();

    // 초기 시퀀스 및 모드별 이벤트 등록
    SetupInitialSequence();
    SetupEventTimeline(GameManager.Instance.gameMode.Value);
}
```

2. Tick 기반 스케줄링

- 코루틴 대신 서버 시간을 비교하는 방식의 스케줄러를 구현하여 프레임 드랍이나 렉 상황에서도 정확한 이벤트 발동을 보장함

```
private void TickScheduler(double now)
{
    for (int i = 0; i < schedule.Count; i++)
    {
        var task = schedule[i];
        // 실행 시간 미달 시 패스
        if (now < task.executeTime) continue;

        // 등록된 델리게이트 실행
        task.action?.Invoke();

        // 반복 작업 갱신 또는 제거
        if (task.repeating) task.executeTime += task.interval;
        else toRemove.Add(i);
    }
}

// 반복 작업 등록 헬퍼 함수
private void AddRepeatingTask(double firstOffsetSeconds, double intervalSeconds, Action action)
{
    schedule.Add(new ScheduledTask
    {
        executeTime = gameStartTimeServer + firstOffsetSeconds,
        interval = intervalSeconds,
        repeating = true,
        action = action
    });
}
```


3. 이벤트 타임라인 및 종료 처리

```
private void SetupEventTimeline(GameManager.GameMode mode)
{
    if (mode == GameManager.GameMode.HideSeek)
    {
        // 20 초 후 첫 미션, 이후 60 초마다 반복
        AddRepeatingTask(20.0, 60.0, () => missionManager.ExecuteRandomMissionServer());
    }

    // 30 초마다 토네이도 생성
    AddRepeatingTask(30.0, 30.0, () => tornadoManager.SpawnOnceServer());
}

[Rpc(SendTo.Everyone)]
private void EndGameRpc(Role winner)
{
    if (IsSessionOwner)
    {
        // 결과 스냅샷 생성 및 전송
        var snapshot = BuildResultSnapshot();
        ShowResultRpc(winner, snapshot);
    }

    schedule.Clear();
    SetGameLoopRpc(false);
    StartCoroutine(ReturnLobbyCo()); // 로비 복귀
}
```

6. 미션 시스템 (MissionManager.cs)

서버 권한으로 미션을 선정하고 브로드캐스팅하여 모든 클라이언트가 동일한 목표를 수행하게 함

```
public void ExecuteRandomMissionServer()
{
    if (!IsSessionOwner) return;

    var index = Random.Range(0, missions.Count);
    // 미션 정보 RPC 로 전송
    AssignMissionRpc((int)mission.type, mission.description, mission.targetValue);
}
```

7. 캐릭터 제어 및 AI 입력 통합 (CharacterBase.cs)

플레이어와 AI가 공유하는 이동 로직을 캡슐화하고 입력 소스만 분리하여 AI 학습 효율을 높임

1. 공통 이동 로직

```
protected void Move(Vector2 dir)
{
    // 소유권 확인 및 상태이상 체크
    if (!IsOwner || isDead.Value || !CanMove) return;

    // 카메라/캐릭터 기준 좌표계로 변환하여 물리 이동
    var moveDir = transform.forward * dir.y + transform.right * dir.x;
    rBody.MovePosition(rBody.position + moveDir * (currentSpeed * Time.fixedDeltaTime));
}
```

2. 입력 소스 분리

```
// [PlayerController.cs] - 플레이어 입력
private void UpdateMovement()
{
    Move(playerInput.MoveInput); // 키보드/마우스 입력 전달
}

// [Npa.cs] - AI 입력
public override void OnActionReceived(ActionBuffers actions)
{
    var dAction = actions.DiscreteActions;
    agent.MoveAction(dAction[0], dAction[1]); // 신경망 판단 결과 전달
}
```

8. 전투 및 물리 상호작용 (HittableBody.cs)

체력 관리와 물리적 낙백 처리를 컴포넌트로 분리하여 재사용성을 높임

1. 독립적인 체력 관리

```
public NetworkVariable<int> healthPoint = new(10);

public void Damaged(int damage, ulong attackerId)
{
    // 소유권자가 아니면 로직 차단 (클라이언트 권한 검증)
    if (!IsOwner) return;

    healthPoint.Value -= damage;
    lastAttackerId = attackerId;
}
```

2. 물리 엔진 기반 낙백 처리

- 행성 중력을 고려한 벡터 연산을 캡슐화하여 일관된 물리 반응을 제공함

```
[Rpc(SendTo.Authority)]
public void KnockBackRpc(Vector3 attackerPos, float power, float upPower)
{
    // 1. 기존 운동량 초기화
    rBody.linearVelocity = Vector3.zero;

    // 2. 행성 중력을 고려한 로컬 '위쪽' 벡터 계산
    var up = -PlanetGravity.Instance.GetGravityDirection(transform.position);

    // 3. 공격자 반대 방향 + 위쪽 방향 합성
    var dir = (transform.position - attackerPos).normalized;
    var finalDir = (dir + up * upPower).normalized;

    // 4. 물리 충격량 적용
    rBody.AddForce(finalDir * power, ForceMode.Impulse);
}
```

[Unity] ML-Agents 기반 자율주행 강화학습 실험

항목	내용
진행 기간	2023.06 ~ 2023.11 (5 개월)
팀 구성	6 명 (팀장)
사용 기술	Unity, ML-Agents, C#, OpenCV
핵심 목표	센서 없이 카메라 입력만으로 작동하는 강화학습 기반 자율주행 구현

기획 배경

- LiDAR 센서 의존도를 낮추고 오직 카메라 영상 처리 데이터만으로 주행 판단을 내리는 시스템을 목표로 함
- 정해진 규칙을 따르는 것이 아니라 시행착오를 통해 최적의 주행 방법을 스스로 학습하는 에이전트 행동 패턴 연구

주요 역할

- 학습 환경 구축: ML-Agents 패키지 연동 및 물리 엔진 튜닝을 통해 에이전트가 주행할 수 있는 시뮬레이션 환경 조성
- 알고리즘 포팅: Python 으로 작성된 OpenCV 차선 인식 알고리즘을 Unity C# 환경에 맞게 이식
- 보상 설계: 에이전트가 차선을 유지하며 주행하는 목표를 달성하도록 Reward Function 설계

프로젝트 성과 및 회고

성과

- Vision-Action 파이프라인 구축: 외부 센서 없이 카메라 데이터 수집부터 전처리 행동 결정으로 이어지는 완전한 자율주행 루프 구현
- 실시간 처리 최적화: 매 프레임 발생하는 텍스처 변환 오버헤드를 효율적으로 관리하여 끊김 없는 학습 환경을 제공함
- 알고리즘 포팅 검증: Python 기반의 OpenCV 로직을 C# 환경에 성공적으로 이식하여 Unity 내에서 독립적인 영상 처리를 실현함

회고 및 개선점

- 환경 설계의 중요성: 강화학습의 성능은 모델 구조보다 정교한 보상 설계와 관측 데이터의 일관성에 더 큰 영향을 받음을 체감함
- 인식 기술의 고도화 필요: 색상 및 엣지 기반의 고전적 방식은 조명 변화와 급격한 곡선에 취약하므로 향후 딥러닝 기반의 세그멘테이션 기술 도입이 필요함
- 학습 전략의 다변화: 단순한 성공 실패 보상 구조로는 이탈 후 복귀하는 고차원 행동을 학습하기 어려우므로 단계적 난이도 설정을 통한 커리큘럼 학습 도입이 요구됨

핵심 구현 상세

1. 차선 인식 파이프라인 (LaneDetect.cs)

Unity 렌더링 화면을 OpenCV 데이터로 변환하고 이미지 전처리를 통해 차선 좌표를 추출함

1. Unity ↔ OpenCV 데이터 변환

- Unity의 RenderTexture는 GPU 메모리에 존재하므로 OpenCV가 처리할 수 있는 CPU 메모리(Texture2D)로 변환

```
// RenderTexture 를 Texture2D 로 변환하는 확장 메서드
public static Texture2D toTexture2D(this RenderTexture rTex)
{
    // RGB24 포맷의 텍스처 생성
    Texture2D tex = new Texture2D(rTex.width, rTex.height, TextureFormat.RGB24, false);

    // 현재 활성화된 렌더 텍스처 백업 및 교체
    var old_rt = RenderTexture.active;
    RenderTexture.active = rTex;

    // GPU -> CPU 픽셀 데이터 읽기
    tex.ReadPixels(new Rect(0, 0, rTex.width, rTex.height), 0, 0);
    tex.Apply();

    // 이전 렌더 텍스처 복구
    RenderTexture.active = old_rt;
    return tex;
}

// 메인 루프에서의 사용
void Update() {
    // 1. 카메라 렌더링 결과 가져오기
    Texture2D img_copy = cam.targetTexture.toTexture2D();

    // 2. OpenCV Mat 객체로 변환
    Mat img_frame = OpenCvSharp.Unity.TextureToMat(img_copy);

    // ... 이후 처리 로직 수행
}
```

2. 시점 변환 (Bird's Eye View) 및 전처리

- 카메라의 원근 왜곡을 제거하여 탑뷰로 변환하고 색상 필터링을 통해 차선 후보를 추출

```
// 1. 관심 영역(ROI) 및 시점 변환 설정
Point[] src_points = {
    new Point(0, height),
    new Point(width * 0.35, height * 0.65),
    new Point(width * 0.65, height * 0.65),
    new Point(width, height)
};

// 변환 행렬 계산 및 적용
Mat M = Cv2.GetPerspectiveTransform(src_points, dst_points);
Cv2.WarpPerspective(img_frame, bird_view, M, new Size(width, height));

// 2. 색상 필터링 (HSV)
Cv2.CvtColor(bird_view, hsv, ColorConversionCodes.BGR2HSV);

// 흰색 차선 범위 마스크
Cv2.InRange(hsv, new Scalar(0, 0, 150), new Scalar(60, 30, 200), white_mask);

// 노란색 차선 범위 마스크
Cv2.InRange(hsv, new Scalar(20, 170, 170), new Scalar(40, 255, 255), yellow_mask);

// 3. 엣지 검출 (Canny Edge)
// 두 마스크를 합친 후 엣지 검출 수행
Cv2.BitwiseOr(white_mask, yellow_mask, combined_mask);
Cv2.Canny(combined_mask, edges, 100, 120);
```

3. 슬라이딩 윈도우 기반 차선 추적

- 히스토그램으로 시작점을 찾고 윈도우를 위로 이동시키며 차선 곡선을 추적

```
// 1. 히스토그램 분석으로 시작점 탐색
Mat cropped = new Mat(edges, new Rect(0, height/2, width, height/2));
Cv2.Reduce(cropped, histogram, ReduceDimension.Row, ReduceTypes.Sum, -1);
Cv2.MinMaxLoc(histogram, out _, out _, out _, out Point maxLoc);

int currentX = maxLoc.X; // 추적 시작 x 좌표

// 2. 슬라이딩 윈도우 루프
int nwindows = 12;
int window_height = height / nwindows;

for(int i = 0; i < nwindows; i++)
{
    // 윈도우 영역 설정
    int win_y_low = height - (i + 1) * window_height;
    int win_y_high = height - i * window_height;
    int win_x_low = currentX - margin;
    int win_x_high = currentX + margin;

    // 윈도우 내의 차선 픽셀(0 이 아닌 값) 수집
    // ... (픽셀 수집 로직)

    // 픽셀이 일정 개수 이상이면 평균 위치로 중심 이동
    if (good_pixels.Count > minpix)
    {
        currentX = (int)good_pixels.Average(p => p.X);
    }

    // 검출된 좌표 저장
    lane_pixels.AddRange(good_pixels);
}
}
```

2. 자율주행 에이전트 설계 (DrivingAgent.cs)

상태 초기화부터 행동 결정, 보상 부여까지 강화학습 에이전트의 전체 수명 주기를 관리함

1. 초기화 및 에피소드 시작

- 학습 안정성을 위해 물리 엔진 파라미터를 조정하고 매 에피소드마다 차량 상태를 리셋

```
public override void Initialize()
{
    // 학습 파라미터 설정
    MaxStep = 10000; // 긴 주행 학습을 위해 충분한 스텝 확보

    rigidbody = GetComponent<Rigidbody>();
    // 무게 중심을 낮춰 코너링 시 전복 방지
    rigidbody.centerOfMass = new Vector3(0, -1f, 0);
}

public override void OnEpisodeBegin()
{
    // 메모리 누수 방지를 위한 리소스 정리
    Resources.UnloadUnusedAssets();

    // 차량 물리 상태 초기화 (속도, 회전 제거)
    rigidbody.velocity = Vector3.zero;
    rigidbody.angularVelocity = Vector3.zero;

    // 출발 위치 및 방향 재설정
    transform.localPosition = new Vector3(1.8f, -4.7f, 0f);
    transform.localRotation = Quaternion.identity;
}
```


2. 행동 결정 (Action Space) 및 보상 부여

- 연속적인 행동 값을 받아 차량을 제어하고 행동 결과에 따라 즉각적인 보상을 부여

```
public override void OnActionReceived(ActionBuffers actions)
{
    // 1. 연속 행동 값 수신 (-1.0 ~ 1.0)
    var continuousActions = actions.ContinuousActions;
    float moveAmount = continuousActions[0]; // 가속/감속
    float steerAmount = continuousActions[1]; // 조향

    // 2. 차량 제어 함수 호출
    Drive(moveAmount);
    SteerVehicle(steerAmount);

    // 3. 주행 보상 계산
    // 일정 속도 이상으로 전진 중일 때 보상 부여 (정지 방지)
    if (moveAmount >= 0.3f)
    {
        // 에피소드 길이에 맞춰 정규화된 보상 추가
        AddReward(1.0f / MaxStep);
    }
    else
    {
        // 너무 느리거나 후진 시 미세한 패널티
        AddReward(-0.1f / MaxStep);
    }

    // 4. 종료 조건 체크 (트랙 이탈)
    if (transform.position.y < -1.0f)
    {
        SetReward(-1.0f); // 실패 패널티
        EndEpisode();     // 학습 종료 및 리셋
    }
}
```

3. 수동 조작 (Heuristic) 및 충돌 처리

- 테스트를 위한 수동 조작 로직과 장애물 충돌 시 처리 로직

```
// 개발자 테스트용 키보드 입력 매핑
public override void Heuristic(in ActionBuffers actionsOut)
{
    var continuousActionsOut = actionsOut.ContinuousActions;
    continuousActionsOut[0] = Input.GetAxis("Vertical"); // W/S 키
    continuousActionsOut[1] = Input.GetAxis("Horizontal"); // A/D 키
}

// 장애물(DeathLine) 충돌 감지
private void OnTriggerEnter(Collider other)
{
    if (other.CompareTag("DeathLine"))
    {
        SetReward(-1.0f); // 강력한 패널티
        EndEpisode(); // 학습 종료
    }
}
```

[Unity] Throw it – 2025 인디 게임어스 게임잼

항목	내용
진행 기간	2025.07.11 ~ 2025.07.13 (3일)
팀 구성	4명 (기획 및 개발)
사용 기술	Unity, C#
핵심 목표	상속 구조를 활용한 확장성 있는 상호작용 시스템 설계 및 게임잼 내 완성

기획 배경

- Bacon – The Game에서 영감을 받아 무엇이든 던져서 목표물에 맞추는 캐주얼 게임을 기획함
- 던지는 물체와 맞는 물체의 조합이 계속 늘어날 것을 대비하여 코드 수정 없이 콘텐츠 확장이 가능한 구조를 설계하고자 함

주요 역할

- 물리 투척 시스템: 마우스 드래그 앤 드롭 입력을 물리적인 힘으로 변환하여 물체를 던지는 로직 구현
- 상호작용 설계: 던져지는 물체와 목표물 간의 충돌 처리를 유연하게 관리하기 위한 상속 기반 프레임워크 설계
- 스테이지 관리: 일반 스테이지와 보너스 스테이지가 순환되는 게임 루프 및 씬 관리 구현

프로젝트 성과 및 회고

성과

- 확장성 확보: Throwable과 Interactable 기반의 상속 구조를 통해 새로운 아이템 추가 시 기존 코드를 수정하지 않는 개방 폐쇄 원칙을 부분적으로 실현함
- 사용자 경험 강화: LineDrawer를 이용한 궤적 시각화와 ZoomView 연출로 조작의 직관성과 게임의 완성도를 높임

회고 및 개선점

- 협업 중심 설계의 중요성: 확장성을 고려한 상속 구조가 오히려 팀원들의 이해를 어렵게 하여 작업 속도를 저하시킴을 확인함
- 과도한 추상화의 부작용: 로직이 여러 클래스로 분산되어 코드 흐름 파악이 어려워졌으며 게임잼 같은 단기 프로젝트에서는 직관적인 코드가 더 효율적임을 깨달음
- 초기화 패턴 정립: 싱글톤 객체의 생명주기 관리 미흡으로 씬 전환 시 객체 중복 생성 문제가 발생했으나 올바른 초기화 패턴으로 해결함

핵심 구현 상세

1. 투척 시스템 (Thrower.cs)

입력 이벤트를 물리적인 힘으로 변환하고 실제 투척 동작은 템플릿 메서드 패턴으로 자식 클래스에 위임함

1. 입력 처리 및 힘 계산

- Update문에서 폴링하지 않고 InputSystem 이벤트를 구독하여 입력 로직을 분리함 드래그 거리에 비례하여 힘을 정규화함

```
// 마우스 드래그 중 호출되는 메서드
protected virtual void OnMouseButtonHold(Vector2 mousePosition)
{
    if (IsThrown) return;

    // 시작 지점과 현재 마우스 위치의 차이 벡터 계산
    var offset = _startPosition - mousePosition;

    // 투척 방향 설정 (정규화된 벡터)
    ThrowDirection = offset.normalized;

    // 드래그 거리에 비례하여 힘 계산
    // 0 ~ 1000 사이의 거리를 0~1로 매핑 후 최대 힘 적용
    ThrowForce = Mathf.InverseLerp(0, 1000, offset.magnitude) * maxThrowForce;
}

// 마우스 버튼을 뗐을 때 호출
protected virtual void OnMouseButtonUp(Vector2 mousePosition)
{
    IsThrown = true;
    // 사운드 재생 및 투척 이벤트 발생
    if (SoundManager.Instance) SoundManager.Instance.Play_R_SFX("sound_throw_", 2);
    OnThrow?.Invoke();
}
```

2. 상호작용 프레임워크 (Interaction)

던져지는 물체와 목표물 간의 결합을 느슨하게 하기 위해 인터페이스 성격의 부모 클래스를 활용함

1. 충돌 처리 위임

- Throwable은 충돌 감지만 담당하고 구체적인 반응 로직은 충돌 대상인 Interactable에게 위임함

```
// Throwable.cs - 던져지는 물체
protected virtual void OnCollisionEnter2D(Collision2D collision)
{
    // 충돌한 물체가 타겟 태그인지 확인
    if (!collision.collider.CompareTag("Target")) return;

    // 상대방의 상호작용 컴포넌트 가져오기
    var interactable = collision.collider.GetComponent<Interactable>();

    // 상대방의 로직 실행 (예: 점수 획득 케이크 장식 등)
    interactable.Interact(collision);

    // 자신의 로직 실행 (예: 물리 효과 정지)
    Interact();
}
```

2. 구체적인 상호작용 구현

- Interactable을 상속받은 클래스에서 Interact 메서드를 오버라이딩하여 각기 다른 게임 기믹을 구현함

```
// Cake.cs - 목표물 구현 예시
public override void Interact(Collision2D collision)
{
    // 충돌 지점 좌표 획득
    var contactPoint = collision.GetContact(0).point;

    // 해당 지점에 크림 생성 효과
    SpawnCream(contactPoint);
}
```

3. 스테이지 순환 관리 (StageManager.cs)

일반 스테이지와 보너스 스테이지를 번갈아 생성하며 게임의 흐름을 제어함

1. 동적 스테이지 생성

- 미리 씬에 객체를 배치하지 않고 프리팹 리스트를 활용해 런타임에 필요한 객체를 생성하여 메모리를 관리함

```
public void StageStart()
{
    // 배경색 랜덤 변경으로 시각적 지루함 방지
    SetRandomCameraBackgroundColor();

    // 짝수 스테이지는 케이크 꾸미기(보너스) 모드로 설정
    if (IsCakeStage() && currentCakeStage < cakeThrowerList.Count)
    {
        cake.gameObject.SetActive(true);
        currentInteractable = cake;
        // 케이크 전용 투척 물체 생성
        currentThrower =
        Instantiate(cakeThrowerList[currentCakeStage]).GetComponent<Thrower>();
    }
    // 홀수 스테이지는 일반 던지기 모드
    else if (_currentStage < throwerList.Count)
    {
        currentInteractable =
        Instantiate(interactableList[_currentStage]).GetComponent<Interactable>();
        currentThrower = Instantiate(throwerList[_currentStage]).GetComponent<Thrower>();
    }
}
```

[Unity] WALL – 2025 게임마당 게임잼

항목	내용
진행 기간	2025.09.05 ~ 2025.09.07 (3일)
팀 구성	4명 (개발)
사용 기술	Unity, C#
핵심 목표	물리 엔진을 활용한 로프 액션과 모듈화된 상호작용 시스템 구현

기획 배경

- 단순 점프를 넘어 로프 반동 및 벽 타기 등 물리 엔진을 적극 활용한 조작감 구현
- 짧은 개발 기간 동안 기획자가 다양한 기믹을 쉽게 배치하고 연결할 수 있는 상호작용 시스템 필요

주요 역할

- 특수 이동 시스템: 파쿠르 및 로프 스윙 등 물리 기반 캐릭터 컨트롤러 구현
- 몬스터 AI: FSM 기반의 상태 패턴 및 외부 장치와 상호작용 가능한 AI 설계
- 트러블 슈팅: 유니티 물리 엔진 관련 최적화 및 안정화 작업

프로젝트 성과 및 회고

성과

- 물리 엔진 제어: HingeJoint2D로 연결된 세그먼트에 플레이어 입력 힘을 가해 그네처럼 반동을 주는 물리적 스윙 구현
- 비동기 상태 관리: 복잡한 클래스 구조 대신 코루틴을 활용하여 몬스터의 순찰 및 추격 상태 전환을 효율적으로 제어함

회고 및 개선점

- 로프 물리 안정화: 고속 이동 시 세그먼트 사이가 벌어지는 현상을 질량 중심 조절로 완화했으나 완벽한 해결을 위해 Verlet Integration 도입이 필요함
- 에디터 UX 고려: 기획 편의를 위해 모든 변수를 노출했으나 오히려 선택지가 많아 혼란을 줌. 개발 도구로서 직관적인 커스텀 에디터 설계의 중요성을 배움

핵심 구현 상세

1. 등반 및 로프 액션 (PlayerClimb.cs)

사다리과 로프의 공통점인 '등반'을 추상화하여 통합하고 로프의 경우 별도의 물리 연산을 추가함

1. 등반 로직 통합

- BaseInteractable을 상속받은 대상을 변수에 할당하여 코드 중복을 최소화함

```
// 사다리와 로프 공용 등반 처리
private void HandleClimbing()
{
    // 등반 중이 아니거나 대상이 없으면 리턴
    if (!IsClimbing || !currentClimbable) return;

    // 수직 입력 값에 따른 이동 처리
    var verticalInput = inputHandler.MoveInput.y;

    // ... 위치 보정 및 애니메이션 동기화 로직
}
```

2. 로프 전용 물리 처리

- 로프에 매달렸을 때는 단순 이동 외에 좌우 입력에 따른 물리적인 힘을 가해 반동을 구현함

```
private void RopeTick()
{
    // 로프 모드일 때만 별도 호출
    var horizontalInput = inputHandler.MoveInput.x;

    // 입력이 있을 경우 로프 세그먼트에 힘 적용
    if (Mathf.Abs(horizontalInput) > 0.01f)
    {
        // 입력 방향으로 힘을 가해 그네와 같은 스윙 효과 구현
        Vector2 force = forceDir * ropeKickImpulse * horizontalInput;
        ropeBody.AddForceAtPosition(force, transform.position);
    }
}
```


2. 몬스터 AI (Monster.cs)

ISwitch 인터페이스를 통해 몬스터를 퍼즐 요소로 활용하고 코루틴으로 상태 패턴을 구현함

1. 인터페이스 기반 상호작용

- 몬스터가 단순한 적을 넘어 버튼이나 레버로 제어되는 퍼즐의 일부로 작동하도록 설계함

```
public class Monster : MonoBehaviour, ISwitch
{
    // 외부 스위치 작동 시 호출되는 인터페이스 구현
    public void OnSwitch()
    {
        // 현재 설정된 타겟 모드로 상태 강제 전환
        switch (targetMode)
        {
            case MonsterMode.Patrol: StartPatrol(); break;
            case MonsterMode.Chase: StartChase(); break;
            // ... 대기 및 공격 모드 전환
        }
    }
}
```

2. 코루틴 기반 FSM

- FixedUpdate의 복잡한 조건문 대신 코루틴을 사용하여 상태별 행동 로직을 직관적으로 분리함

```
private IEnumerator PatrolCo()
{
    // 순찰 모드인 동안 반복 실행
    while (mode == MonsterMode.Patrol)
    {
        // 지정된 웨이포인트 사이를 왕복 이동
        MoveToNextWaypoint();

        // 도착 후 일정 시간 대기
        yield return waitTime;

        // 다음 지점으로 목표 갱신
        UpdateTargetPoint();
    }
}
```

[Web] Hanbat Box - AWS S3 기반 파일 공유 서비스

항목	내용
진행 기간	2024.10.07 ~ 2024.11.10 (약 1개월)
팀 구성	3명 (백엔드)
사용 기술	AWS, S3, Spring Boot
핵심 목표	교내에서 간단하게 파일을 공유할 수 있는 웹 서비스 개발
수상 실적	 교내 2024년 소중한 오픈소스 활용 SW경진대회 은상
링크	https://github.com/west-eastH/Hanbat-Box

기획 배경

- 미경험 분야인 웹 백엔드 기술과 클라우드 스토리지 환경을 실제로 적용해보기 위해 **교내 대회에 출품하는 프로젝트**로 기획함
- 실무에서 활용되는 기술 스택을 경험하는 것을 목표로 함
- 교내 구성원 간 원활한 자료 공유를 통한 **작업 능력** 및 **교육 품질 향상**을 목표로 함

주요 역할

- **Spring Boot** 기반 서버와 **AWS S3** 간 **파일 업로드 및 다운로드** 기능을 구현하고 파일 메타데이터 기반 응답 구조 설계
- ResponseEntity와 HttpHeaders를 활용하여 **Content-Disposition** 기반 **다운로드 응답 처리** 구현
- 파일 식별자 기반 조회 구조를 설계하고 서비스 계층에서 S3 객체 조회 및 반환 흐름 구성

프로젝트 성과 및 회고

성과

- 클라우드 스토리지 활용 경험 및 파일 업로드.조회 프로세스 습득
- 공식 문서 기반의 자기주도적 문제 해결 및 학습 프로세스 경험
- **교내 경진 대회 은상**

회고 및 개선점

- 사전 학습 부족으로 인해 S3의 활용 범위가 기본 기능 구현에 머무른 점이 아쉬움
- 향후 IAM 정책, 보안 설정, 성능 최적화 등 심화 내용을 중심으로 추가 학습이 필요함

핵심 구현 상세

1. Box-Item 기반 파일 공유 데이터 구조 설계

하나의 공유 단위인 Box에 여러 개의 파일 Item을 연결하는 구조로 설계

```
@Entity
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Box {

    @Id
    @GeneratedValue
    @Column(name = "box_id")
    private Long id;

    private String uploader;
    private String title;
    private String password;
    private Long fileSize;
    private LocalDateTime dateUploaded;
    private boolean crypted;

    @OneToMany(mappedBy = "box", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Item> items = new ArrayList<>();
}
```

```
@Entity
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
@Table(name = "item")
public class Item {

    @Id
    @GeneratedValue
    @Column(name = "item_id")
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "box_id")
    private Box box;

    private String objectKey;
}
```

코드 설명

- 하나의 공유 링크에서 여러 파일을 관리할 수 있도록 Box와 Item을 1:N 관계로 분리
- 실제 파일은 S3에 저장하고, DB에는 S3 접근 키인 objectKey만 저장
- 공유 단위의 제목, 업로더, 암호화 여부, 전체 파일 크기를 Box에서 관리

2. S3 Object Key 생성 및 업로드 요청 구조 구현

파일 업로드 시 원본 파일명을 그대로 사용하지 않고 시간 기반 prefix와 제목을 조합하여 S3 Object Key를 생성

```
private String createObjectKey(String title, String originFileName) {  
    if (originFileName == null || !originFileName.contains(".")) {  
        throw new IllegalArgumentException("올바른 파일을 추가해 주세요.");  
    }  
  
    String extension =  
        originFileName.substring(originFileName.lastIndexOf('.') + 1);  
  
    return "%d-%s.%s".formatted(  
        System.currentTimeMillis(),  
        title,  
        extension  
    );  
}
```

S3 업로드 요청은 파일 크기와 Content-Type 을 포함해 생성

```
private PutObjectRequest buildPutObjectRequest(  
    String objectKey,  
    MultipartFile file  
) {  
    return PutObjectRequest.builder()  
        .bucket(bucketName)  
        .key(objectKey)  
        .contentType(file.getContentType())  
        .build();  
}
```

코드 설명

- S3 객체 식별을 위한 고유 Object Key 생성
- 파일 확장자 유효성 검증
- S3 저장 시 파일 크기와 MIME Type을 함께 전달하여 파일 메타데이터 유지

3. AWS S3 TransferManager 기반 파일 업로드 구현

AWS SDK의 S3TransferManager를 사용하여 서버에서 S3로 파일을 업로드하는 통신 모듈을 구현

```
public String uploadFileAndGetObjectKey(
    String title,
    MultipartFile file
) throws IOException {

    String objectKey =
        createObjectKey(
            title,
            Objects.requireNonNull(file.getOriginalFilename())
        );

    BlockingInputStreamAsyncRequestBody body =
        AsyncRequestBody.forBlockingInputStream(file.getSize());

    transferManager.upload(
        buildUploadRequest(objectKey, file, body)
    );

    body.writeInputStream(
        new ByteArrayInputStream(file.getBytes())
    );

    return objectKey;
}
```

코드 설명

- 서버 애플리케이션과 S3 사이의 파일 업로드 흐름을 서비스 계층으로 분리
- 업로드 후 DB에는 실제 파일이 아닌 objectKey만 저장하여 파일 저장소와 비즈니스 데이터를 분리
- 업로드 성공 시 이후 다운로드에 사용할 S3 Object Key 반환

4. 파일 업로드와 Box-Item 저장 흐름 구현

Box 생성 시 비밀번호를 암호화하고, 여러 파일을 각각 S3에 업로드한 뒤 Item으로 연결

```
public Long saveBoxWithItems(
    BoxSaveDto boxDto,
    List<MultipartFile> files
) {
    String encodedPassword = null;
    boolean isCrypted = false;

    if (boxDto.getPassword() != null && !boxDto.getPassword().isEmpty()) {
        encodedPassword = passwordEncoder.encode(boxDto.getPassword());
        isCrypted = true;
    }

    Box box = Box.createBox(
        boxDto.getUploader(),
        boxDto.getTitle(),
        encodedPassword,
        isCrypted
    );
}
```

```
files.forEach(file -> {

    String objectKey;

    try {
        objectKey = s3Service.uploadFileAndGetObjectKey(
            boxDto.getTitle(),
            file
        );
    } catch (IOException e) {
        throw new RuntimeException(e);
    }

    Item item = Item.createItem(objectKey);

    box.addItem(item, file.getSize());
});

boxRepository.save(box);

return box.getId();
```

코드 설명

- 공유 단위 생성과 파일 업로드 흐름을 하나의 서비스 로직으로 연결
- 비밀번호가 있는 경우만 암호화 처리하여 공개/비공개 공유를 모두 지원
- 여러 파일 업로드 시 Box 하나에 여러 Item을 연결하여 다운로드 시 단일 공유 단위로 처리

5. 비밀번호 기반 파일 접근 제어 구현

파일 다운로드 또는 삭제 요청 시 비밀번호가 설정된 Box인지 확인하고, 암호화된 비밀번호와 입력값을 비교

```
public boolean validatePassword(Long id, String inputPassword) {
    Box box = boxRepository.findById(id)
        .orElseThrow(() ->
            new ResponseStatusException(
                HttpStatus.NOT_FOUND,
                "Box not found"
            )
        );

    if (box.isCrypted()) {
        return passwordEncoder.matches(
            inputPassword,
            box.getPassword()
        );
    }

    return true;
}
```

다운로드 요청에서는 검증 실패 시 즉시 예외를 발생

```
public S3FileDetails downloadFile(Long id, String password) {
    if (!validatePassword(id, password)) {
        throw new ResponseStatusException(
            HttpStatus.UNAUTHORIZED,
            "Invalid password"
        );
    }

    Box box = boxRepository.findById(id)
        .orElseThrow(() ->
            new ResponseStatusException(
                HttpStatus.NOT_FOUND,
                "Box not found"
            )
        );
}
```

코드 설명

- 공유 파일에 선택적 비밀번호 보호 기능 제공
- DB에는 평문 비밀번호가 아닌 인코딩된 값 저장
- 다운로드와 삭제 요청 모두 동일한 검증 로직 재사용

6. 단일 파일 다운로드와 다중 파일 ZIP 다운로드 분기 처리

Box에 포함된 파일 개수에 따라 단일 파일 다운로드와 ZIP 다운로드를 분기

```
public S3FileDetails downloadFile(Long id, String password) {
    if (!validatePassword(id, password)) {
        throw new ResponseStatusException(
            HttpStatus.UNAUTHORIZED,
            "Invalid password"
        );
    }

    Box box = boxRepository.findById(id)
        .orElseThrow(() ->
            new ResponseStatusException(
                HttpStatus.NOT_FOUND,
                "Box not found"
            )
        );

    String[] objectKeys = getObjectKeysByBoxId(id);

    if (isSingleFile(objectKeys)) {
        return downloadSingleFile(
            objectKeys[0],
            objectKeys[0],
            box.getFileSize()
        );
    }

    String title = getBoxTitleById(id) + ".zip";

    return downloadMultipleFiles(title, objectKeys);
}
```

```
private boolean isSingleFile(String[] keys) {
    return keys.length == 1;
}
```

코드 설명

- 단일 파일은 원본 파일 형태로 바로 다운로드
- 여러 파일은 하나의 ZIP 파일로 묶어 공유 사용성을 개선
- 클라이언트는 동일한 다운로드 API를 사용하고, 서버가 내부적으로 파일 개수에 따라 응답 형태 결정

7. S3 파일 다운로드 및 byte[] 응답 구성

S3 Object Key를 기반으로 파일을 다운로드하고, 응답에 사용할 파일명·크기·내용을 DTO로 구성

```
private DownloadRequest<ResponseBytes<GetObjectResponse>>
buildDownloadRequest(String objectKey) {

    return DownloadRequest.builder()
        .getObjectRequest(buildGetObjectRequest(objectKey))
        .responseTransformer(AsyncResponseTransformer.toBytes())
        .build();
}
```

```
public byte[] downloads(String objectKey) {
    return transferManager
        .download(buildDownloadRequest(objectKey))
        .completionFuture()
        .orTimeout(3, TimeUnit.MINUTES)
        .join()
        .result()
        .asByteArray();
}
```

```
private S3FileDetails downloadSingleFile(
    String title,
    String objectKey,
    Long fileSize
) {
    byte[] bytes = downloads(objectKey);

    return new S3FileDetails(
        title,
        fileSize,
        bytes
    );
}
```

코드 설명

- S3 다운로드 요청 생성 로직을 별도 메서드로 분리
- 다운로드 제한 시간을 설정하여 장시간 대기 방지
- 파일명, 파일 크기, 파일 내용을 하나의 응답 DTO로 묶어 Controller에서 응답 생성에 활용

8. 다중 파일 ZIP 압축 다운로드 구현

여러 개의 S3 파일을 다운로드한 뒤 서버에서 ZIP 파일로 묶어 반환

```
private S3FileDetails downloadMultipleFiles(  
    String title,  
    String[] objectKeys  
) {  
    try (  
        ByteArrayOutputStream byteOutput =  
            new ByteArrayOutputStream();  
  
        ZipArchiveOutputStream zipOut =  
            new ZipArchiveOutputStream(byteOutput)  
    ) {  
        for (String objectKey : objectKeys) {  
            byte[] bytes = downloads(objectKey);  
  
            ZipArchiveEntry zipEntry =  
                new ZipArchiveEntry(objectKey);  
  
            zipOut.putArchiveEntry(zipEntry);  
  
            zipOut.write(bytes);  
  
            zipOut.closeArchiveEntry();  
        }  
  
        zipOut.finish();  
  
        return new S3FileDetails(  
            title,  
            byteOutput.size(),  
            byteOutput.toByteArray()  
        );  
    }  
}
```

코드 설명

- 여러 파일을 각각 다운로드하지 않고 하나의 ZIP 파일로 제공
- Box 제목을 ZIP 파일명으로 사용하여 공유 단위의 의미 유지
- ZIP 생성 실패 시 서버 오류로 처리하여 다운로드 실패를 명확히 전달

9. 다운로드 응답 헤더 구성

다운로드 응답 시 브라우저가 파일로 인식할 수 있도록 Content-Disposition, Content-Length, Content-Type 헤더를 구성

```
public static HttpHeaders getResponseHeader(String title, long size) {
    HttpHeaders headers = new HttpHeaders();

    String encodedTitle = URLEncoder.encode(title, StandardCharsets.UTF_8);

    headers.setContentType(MediaType.APPLICATION_OCTET_STREAM);
    headers.setContentLength(size);

    headers.add(
        HttpHeaders.CONTENT_DISPOSITION,
        "attachment; filename=" + encodedTitle
    );

    headers.add(
        "Content-Transfer-Encoding",
        "binary"
    );

    return headers;
}
```

Controller에서는 해당 헤더와 파일 내용을 함께 반환

```
@PostMapping("/downloads/{id}")
public ResponseEntity<?> downloads(
    @PathVariable("id") Long id,
    @RequestBody Password body
) {
    S3FileDetails fileDetails =
        s3Service.downloadFile(id, body.password());

    HttpHeaders responseHeader =
        DownloadResponseBuilder.getResponseHeader(
            fileDetails.fileName(),
            fileDetails.size()
        );

    return new ResponseEntity<>(
        fileDetails.content(),
        responseHeader,
        HttpStatus.OK
    );
}
```

코드 설명

- 다운로드 시 파일명이 깨지지 않도록 UTF-8 인코딩 처리
- 브라우저에서 파일 다운로드 동작이 발생하도록 attachment 헤더 설정
- 단일 파일과 ZIP 파일 모두 동일한 응답 구조로 처리

[Web] Dyslexia - 생성형 AI 기반 난독증 아동 교재 생성 서비스

항목	내용
진행 기간	2025.03 ~ 2025.07 (4개월)
팀 구성	5명 (백엔드)
사용 기술	AWS, Spring Boot, AI
핵심 목표	생성형 AI를 활용해 난독증 아동 맞춤형 교재를 자동 생성하는 서비스 파이프라인 구축
수상 실적	 교내 2025 소중한 SW•AI 경진대회 SW부문 1등
링크	https://github.com/orgs/semitation/repositories

기획 배경

- 난독증 아동이 겪는 텍스트 인식 부담 해소 및 부족한 전용 학습 콘텐츠 확충 필요성 절감
- 맞춤형 교재 제작에 소요되는 높은 시간과 비용을 생성형 AI를 활용해 혁신적으로 절감

주요 역할

- Document → Textbook → Page 구조의 교재 생성 흐름을 정의하고 페이지 단위 학습 데이터를 저장하기 위한 관계형 데이터베이스 모델 설계
- PDFBox를 활용한 페이지 단위 텍스트를 추출하는 파싱 로직 구현
- 교재 및 페이지 데이터로 변환되는 백엔드 처리 흐름 설계
- 교재 생성, 저장, 조회 기능을 담당하는 Spring Boot 기반 REST API 설계 및 구현
- 생성형 AI를 통한 영어 -> 한국어 번역 프롬프트 설계

프로젝트 성과 및 회고

성과

- 사용자 특성 이해가 설계의 핵심임을 깨닫고 UI/UX가 사용자 경험에 미치는 영향 확인
- 기술적 완성도를 넘어 문제 정의와 기획 과정의 중요성을 깊이 인식
- 교내 경진 대회 1등 및 SW 인재 페스티벌 참여

회고 및 개선점

- 실제 타겟인 난독증 아동 대상의 충분한 사용자 테스트를 진행하지 못한 점이 아쉬움
- 학습 효과를 정량적으로 검증할 수 있는 데이터 수집 및 분석 구조의 고도화 필요

핵심 구현 상세

1. 문서 업로드 기반 교재 생성 파이프라인 설계

PDF 업로드 이후 Document → Textbook → Page 구조로 이어지는 교재 생성 흐름을 설계하고 구현
문서 업로드 요청 시 Document 엔티티를 생성하여 파일 메타데이터를 저장하고 이후 교재 생성 단계로 연결되는 구조를 구성

```
@Transactional
public DocumentDto uploadDocument(
    Long guardianId,
    MultipartFile file,
    String title
) throws IOException {

    Guardian guardian =
        guardianRepository.findById(guardianId)
            .orElseThrow(
                () -> new ApplicationException(
                    ExceptionCode.GUARDIAN_NOT_FOUND
                )
            );

    Document document =
        Document.builder()
            .guardian(guardian)
            .title(title)
            .originalFilename(
                file.getOriginalFilename()
            )
            .filePath(
                "temp_" + System.currentTimeMillis()
            )
            .fileSize(file.getSize())
            .mimeType(file.getContentType())
            .build();

    document =
        documentRepository.save(document);
}
```

코드 설명

- 파일 저장 이전 Document 엔티티를 생성하여 파일 메타데이터 관리 구조 확보
- Document 중심 교재 생성 파이프라인 구성
- 이후 Textbook 및 Page 생성 흐름과 연결 가능한 구조 설계

처리 흐름

- PDF 업로드 → S3 저장 → Document 생성 → 페이지 단위 파싱 → Textbook 생성

2. AWS S3 기반 PDF 저장 구조 구현

파일을 로컬 서버가 아닌 S3에 저장하여 문서 처리 시스템과 파일 저장 시스템을 분리

```
String s3Key =
    String.format(
        "%d/%d/%s",
        guardianId,
        documentId,
        uniqueFilename
    );

ObjectMetadata metadata =
    new ObjectMetadata();

metadata.setContentLength(
    file.getSize()
);

metadata.setContentType(
    file.getContentType()
);

metadata.setContentDisposition(
    "inline; filename=\"" +
    uniqueFilename +
    "\""
);

amazonS3Client.putObject(
    bucketName,
    s3Key,
    file.getInputStream(),
    metadata
);
```

코드 설명

- 사용자 / 문서 기준 계층형 저장 구조 설계
- 문서 식별자 기반 파일 접근 구조 구성
- 이후 페이지 단위 분석 및 재처리 시 재사용 가능 구조 확보

3. PDF 페이지 단위 텍스트 추출 파싱 로직 구현

PDF 전체 문서를 하나의 텍스트로 처리하지 않고 페이지 단위 학습 콘텐츠로 활용하기 위한 파싱 구조 구현

```
URL url =
    new URL(s3Url);

try (
    InputStream inputStream =
        url.openStream()
) {

    document =
        PDDocument.load(
            inputStream
        );
}

PDFTextStripper stripper =
    new PDFTextStripper();

int pageCount =
    document.getNumberOfPages();

for (int i = 1;
     i <= pageCount;
     i++) {

    stripper.setStartPage(i);

    stripper.setEndPage(i);

    String pageText =
        stripper.getText(
            document
        );

    pages.add(
        pageText
    );
}
```

코드 설명

- 페이지 단위 콘텐츠 생성 구조 지원
- 페이지 단위 분석 및 확장성 구조 확보
- 이후 학습 콘텐츠 저장 구조와 연결 가능하도록 설계

4. 교재 생성 데이터 모델 설계

Document → Textbook → Page 구조로 이어지는 교재 생성 데이터 모델 설계

설계 구조

- Document
 - └ Textbook
 - └ Page

```
Textbook textbook =
    Textbook.builder()
        .document(document)
        .guardian(
            document.getGuardian()
        )
        .title(
            document.getTitle()
        )
        .pageCount(pageCount)
        .learnRate(0)
        .build();

textbook.setConvertProcessStatus(
    ConvertProcessStatus.PROCESSING
);

textbook =
    textbookRepository.save(
        textbook
    );
```

코드 설명

- Document 단위 파일 저장 구조와 교재 단위 콘텐츠 구조 분리
- 페이지 개수 기반 학습 콘텐츠 관리 구조 설계
- 교재 변환 상태 관리 구조 확보

5. 교재 생성 REST API 설계 및 구현

PDF 업로드 이후 교재 생성까지 이어지는 REST API 흐름 구현

```
@PostMapping(  
    value =  
        "/documents/upload",  
    consumes =  
        MediaType.MULTIPART_FORM_DATA_VALUE  
)  
  
public ResponseEntity<  
    CommonResponse<DocumentDto>  
> uploadDocument(  
  
    @RequestParam("guardianId")  
    Long guardianId,  
  
    @RequestParam("file")  
    MultipartFile file,  
  
    @RequestParam("title")  
    String title  
) throws IOException {  
  
    DocumentDto dto =  
        convertProcessService  
            .uploadDocument(  
                guardianId,  
                file,  
                title  
            );  
  
    return ResponseEntity.ok(  
        new CommonResponse<>(  
            "PDF 업로드 완료", dto  
        )  
    );  
}
```

코드 설명

- MultipartFile 기반 파일 업로드 처리 구조 설계
- 파일 확장자 검증 구조 적용
- 서비스 계층 중심 책임 분리 구조 설계
- 교재 생성 파이프라인과 API 흐름 연결

[Web] UCSI University Internship - 강의실 배정 최적화 시스템

항목	내용
진행 기간	2025.02.16 ~ 2025.03.02
팀 구성	6명 (팀장)
사용 기술	Django Rest Framework, PostregSQL, React / Vlte
핵심 목표	강의실 중복 문제해결을 위한 서비스 개발
링크	https://github.com/mjmj1/UCSI_Subject_Team1

기획 배경

- 고질적인 문제인 강의실 배정 중복 및 시간표 충돌 해결을 위함
- 빠른 프로토타이핑을 위해 Django Rest Framework와 PostgreSQL을 활용한 API 중심 구조 설계

주요 역할

- 전체 일정 관리 및 팀원별 역할 분배, 개발 방향성 수립
- 현지 PM 및 팀원 간의 원활한 소통을 주도하여 **요구사항을 기술적으로 구체화**
- 강의실, 강의, 시간표 간의 복잡한 상관관계를 정의하는 **RDB 스키마 설계**
- Django REST Framework 기반 **API 중심 서비스 구조 설계 및 구현**

프로젝트 성과 및 회고

성과

- 데이터 구조 설계가 복잡한 비즈니스 로직 해결의 핵심임을 실증적으로 체감
- API 중심 설계와 명확한 문서화를 통해 팀 간 역할 분리 및 개발 속도 향상 달성
- 논리적 데이터 모델링을 통해 강의실 중복 문제를 시스템적으로 차단

회고 및 개선점

- 실제 대학 운영 데이터와의 연동 테스트 기회가 부족했던 점이 아쉬움
- 대규모 트래픽 및 대량 데이터 환경에서의 성능 검증 및 최적화 프로세스 보완 필요

핵심 구현 상세

1. 강의·강의실·배정 결과 데이터 모델 설계

강의실 배정 문제를 시스템적으로 처리하기 위해 강의 정보, 강의실 정보, 배정 결과, 미배정 데이터를 각각 분리하여 모델링

```
class FatherCourseOffer(models.Model):
    CourseCode = models.CharField(max_length=20)
    FacultyCode = models.CharField(max_length=20)
    Session = models.CharField(max_length=255)
    Capacity = models.IntegerField()
    MinPerSession = models.IntegerField()
    Lecturer = models.CharField(max_length=255)
    CourseType = models.CharField(max_length=10)
```

```
class ResourceRoom(models.Model):
    ResourceCode = models.CharField(max_length=100)
    Description = models.CharField(max_length=255)
    Capacity = models.IntegerField()
    Lecture = models.CharField(max_length=1)
    Tutorial = models.CharField(max_length=1)
    Lab = models.CharField(max_length=1)
    ETC = models.CharField(max_length=1)
```

```
class AssignTable(models.Model):
    CourseCode = models.CharField(max_length=20)
    ResourceCode = models.CharField(max_length=100)
    FacultyCode = models.CharField(max_length=20)
    Session = models.CharField(max_length=255)
    Lecturer = models.CharField(max_length=255)
    DayOfWeek = models.CharField(max_length=10)
    RoomCapacity = models.IntegerField()
    CourseCapacity = models.IntegerField()
    StartTime = models.IntegerField()
    MinPerSession = models.IntegerField()
```

코드 설명

- MultipartFile 기반 파일 업로드 처리 구조 설계
- 파일 확장자 검증 구조 적용
- 서비스 계층 중심 책임 분리 구조 설계
- 교재 생성 파이프라인과 API 흐름 연결

2. Course Offer 엑셀 데이터 전처리 및 부모·자식 강의 분리

UCSI 측에서 제공한 강의 개설 엑셀 데이터를 읽고, Combined To 구조를 기준으로 부모 강의와 자식 강의를 분리

```
def make_course_tables():
    file_path = os.path.join(settings.BASE_DIR, 'datas', COURSEOFFERPATH)

    df = pd.read_excel(file_path)
    df = df.drop(df.tail(1).index).drop(columns=['Unnamed: 0'])
    df = add_check_code(df)
    df = del_only_combined_to(df)
    df = del_self_reference(df)

    make_family_tables(df)
```

Session 컬럼 안의 Combined To 값을 분석하여 연결 대상 강의를 추출

```
def add_check_code(df):
    df['check_code'] = df['Session'].apply(
        lambda x:
            "Only Combined To" if isinstance(x, str) and x.strip() == "Combined To /"
            else x.split('Combined To ')[-1].split('/', 1)[0]
            if isinstance(x, str) and 'Combined To' in x
            else None
    )
    df['FatherCode'] = None

    return df
```

부모 강의와 자식 강의를 분리한 뒤 각각 테이블에 저장

```
def separate_table(df):
    father_list = []
    child_list = []

    for index, row in df.iterrows():
        if row['check_code'] is None:
            father_list.append(row)
        else:
            child_list.append(row)

    father_df = pd.DataFrame(father_list)
    child_df = pd.DataFrame(child_list)

    father_df.drop(columns=['check_code', 'FatherCode'], inplace=True)

    return father_df, child_df
```

코드 설명

- 병합 강의 정보를 단순 문자열로 두지 않고 관계형 데이터로 분리
- 부모 강의 기준으로 최종 배정하고, 자식 강의의 수강 인원을 합산할 수 있는 구조 확보
- 예외 데이터는 별도 테이블로 분리하여 데이터 정합성 유지

3. Resource Room 엑셀 데이터 전처리 및 강의실 유형 태깅

강의실 데이터를 읽어 수용 인원과 사용 가능 유형을 저장, 일부 특수 강의실은 ResourceCode 키워드를 기준으로 ETC 태그를 자동 부여

```
def make_resource_table():
    file_path = os.path.join(settings.BASE_DIR, 'datas', RESOURCEROOMPATH)

    df = pd.read_excel(file_path)

    df = add_room_tag(df)

    df.fillna({
        'Lecture': 'N',
        'Tutorial': 'N',
        'Lab': 'N',
        'ETC': 'N',
    }, inplace=True)
```

```
def add_room_tag(df):
    keywords = ['imus', 'kitchen', 'pbl']

    df['ETC'] = df['Resource Code'].apply(
        lambda x: 'Y'
        if any(keyword in str(x).lower() for keyword in keywords)
        else 'N'
    )

    return df
```

강의실 데이터는 ResourceRoom 모델로 일괄 저장

```
for _, row in df.iterrows():
    data = ResourceRoom(
        ResourceCode=row['Resource Code'],
        Description=row['Description'],
        Capacity=row['Capacity'],
        Lecture=row['Lecture'],
        Tutorial=row['Tutorial'],
        Lab=row['Lab'],
        ETC=row['ETC'],
    )

    data_list.append(data)

ResourceRoom.objects.bulk_create(data_list)
```

코드 설명

- 강의실을 단순 공간이 아니라 Lecture, Tutorial, Lab, ETC 유형별 자원으로 관리
- 강의 유형과 강의실 유형을 매칭할 수 있는 기반 데이터 구성
- 엑셀 원본의 결측값을 기본값으로 보정하여 배정 로직 오류 방지

4. 장시간 강의 분할 처리

한 강의 시간이 180분을 초과할 경우 배정 가능한 시간 단위로 분할하여 시간표 생성 로직에 전달

```
def split_min_per_session(df):
    new_rows = []
    split_id_counter = 1

    for idx, row in df.iterrows():
        min_per_session = row["MinPerSession"]

        if min_per_session > 180:
            split_label = str(split_id_counter)
            split_id_counter += 1
        else:
            split_label = "N"

        first_split = True

        while min_per_session > 180:
            new_row = row.copy()
            new_row["MinPerSession"] = 180
            new_row["Split"] = "Y"
            new_row["Split_ID"] = split_label
            new_rows.append(new_row)

            min_per_session -= 180
            first_split = False

        new_row = row.copy()
        new_row["MinPerSession"] = min_per_session
        new_row["Split"] = "Y" if not first_split else "N"
        new_row["Split_ID"] = split_label
        new_rows.append(new_row)

    return pd.DataFrame(new_rows).reset_index(drop=True)
```

코드 설명

- 긴 강의를 그대로 배정하면 시간 슬롯 충돌 가능성이 커지므로 일정 단위로 분할
- 동일 강의에서 분리된 세션은 Split_ID로 묶어 중복 배정 여부 확인 가능
- 시간표 생성 시 교수 중복, 강의 중복, 시간 중복을 더 세밀하게 제어

5. 교수·강의 분할 조건을 고려한 시간 슬롯 생성

시간표 후보를 생성할 때 교수별 스케줄과 분할 강의 스케줄을 별도로 관리하여 같은 시간대에 중복 배정되지 않도록 처리

```
professor_schedule = {}
split_schedule = {}

for _, lecture in course_df.iterrows():
    professor = lecture["Lecturer"]
    split_id = lecture["Split_ID"] if lecture["Split"] == "Y" else None
    session_duration = lecture['MinPerSession'] / 60

    if professor not in professor_schedule:
        professor_schedule[professor] = []

    if split_id and split_id not in split_schedule:
        split_schedule[split_id] = []
```

가능한 시간 슬롯 필터링

```
available_slots = []

for d in days:
    for t in times:
        end_t = t + session_duration

        professor_conflict = any(
            scheduled_day == d and not (
                end_t <= scheduled_start or t >= scheduled_end
            )
            for scheduled_day, scheduled_start, scheduled_end
            in professor_schedule[professor]
        )

        split_conflict = split_id and any(
            scheduled_day == d and not (
                end_t <= scheduled_start or t >= scheduled_end
            )
            for scheduled_day, scheduled_start, scheduled_end
            in split_schedule[split_id]
        )

        if not professor_conflict and not split_conflict and end_t <= 22:
            available_slots.append((d, t, end_t))
```

코드 설명

- 같은 교수가 동일 시간대에 여러 강의를 맡는 문제 방지
- 분할된 강의가 같은 시간대에 겹치는 문제 방지
- 22시 이후 종료되는 강의 배정 방지

6. 강의실 수용 인원·강의 유형 기반 배정 로직

강의 유형과 수강 인원에 맞는 강의실 후보를 먼저 필터링한 뒤, 시간 충돌이 없는 강의실을 선택

```
def get_available_rooms(capacity_needed, course_type=None):
    if course_type:
        if course_type == 'Le':
            available_rooms = rooms_df[
                (rooms_df['Capacity'] >= capacity_needed) &
                (rooms_df['Lecture'] == 'Y')
            ]
        elif course_type == 'La':
            available_rooms = rooms_df[
                (rooms_df['Capacity'] >= capacity_needed) &
                (rooms_df['Lab'] == 'Y')
            ]
        elif course_type == 'Tu':
            available_rooms = rooms_df[
                (rooms_df['Capacity'] >= capacity_needed) &
                (rooms_df['Tutorial'] == 'Y')
            ]
        elif course_type == 'ETC':
            available_rooms = rooms_df[
                rooms_df['Capacity'] >= capacity_needed
            ]
        else:
            return pd.DataFrame()
    else:
        available_rooms = rooms_df[
            rooms_df['Capacity'] >= capacity_needed
        ]
```

수용 인원 낭비가 적은 강의실을 우선 선택

```
available_rooms = available_rooms.copy()
available_rooms["excess_capacity"] = (
    available_rooms["Capacity"] - capacity_needed
)

available_rooms = available_rooms.sort_values(
    by=["excess_capacity", "Capacity"],
    ascending=[True, True]
)
```

코드 설명

- 강의 수강 인원보다 작은 강의실 배정 방지
- Lecture, Lab, Tutorial 등 강의 유형에 맞는 공간 매칭
- 수용 인원 낭비를 줄이기 위해 초과 좌석이 적은 강의실 우선 선택

7. 강의실 시간 충돌 방지 및 사용 빈도 기반 선택

강의실별 사용 시간을 별도로 관리하여 같은 강의실이 같은 시간에 중복 배정되지 않도록 함

```
room_schedule = {}
room_usage_count = {room: 0 for room in rooms_df['ResourceCode']}
```

```
def assign_lecture(lecture, available_rooms):
    non_conflicting_rooms = []

    day = lecture['day']
    start_time = lecture['start_time']
    end_time = lecture['end_time']

    for _, room in available_rooms.iterrows():
        room_name = room['ResourceCode']

        if room_name not in room_schedule:
            room_schedule[room_name] = []

        conflict = any(
            scheduled_day == day and not (
                end_time <= scheduled_start or
                start_time >= scheduled_end
            )
            for scheduled_day, scheduled_start, scheduled_end
            in room_schedule[room_name]
        )

        if not conflict:
            non_conflicting_rooms.append(room)
```

중복이 없는 강의실 중 사용 빈도가 낮은 강의실을 우선 선택

```
if non_conflicting_rooms:
    non_conflicting_rooms = sorted(
        non_conflicting_rooms,
        key=lambda x: room_usage_count[x['ResourceCode']]
    )

    selected_room = non_conflicting_rooms[0]
    room_name = selected_room['ResourceCode']

    room_schedule[room_name].append((day, start_time, end_time))

    room_usage_count[room_name] += 1

    return room_name, selected_room['Capacity']
```

코드 설명

- 강의실 중복 배정 방지
- 특정 강의실에 배정이 물리는 현상 완화
- 가능한 강의실 중 시간 충돌이 없는 강의실만 선택

8. 제약 조건 검증 및 최적 시간표 선택

생성된 시간표 후보에 대해 강의실 중복, 교수 중복, 수용 인원 초과, 분할 강의 충돌을 검사하고 가장 제약 위반이 적은 시간표를 선택

```
def check_constraints(final_timetable):
    constraint_violations = []

    room_conflicts = {}

    for idx, row in final_timetable.iterrows():
        room = row['room']
        day = row['day']
        start_time = row['start_time']
        end_time = row['end_time']

        if room:
            if room not in room_conflicts:
                room_conflicts[room] = []

            for scheduled_day, scheduled_start, scheduled_end in room_conflicts[room]:
                if scheduled_day == day and not (
                    end_time <= scheduled_start or
                    start_time >= scheduled_end
                ):
                    constraint_violations.append(
                        f"Room Conflict: {room}"
                    )

            room_conflicts[room].append(
                (day, start_time, end_time)
            )
```

```
def evaluate_timetable(timetable_result):
    final_timetable, unused_rooms, unassigned_courses = timetable_result
    violations = check_constraints(final_timetable)

    return len(violations) + len(unused_rooms)
```

```
best_timetable = min(
    results,
    key=evaluate_timetable
)
```

코드 설명

- 단일 시간표를 바로 확정하지 않고 여러 후보를 생성한 뒤 비교
- 제약 위반 수와 미사용 강의실 수를 기준으로 후보 평가
- 최종 결과의 충돌 가능성을 줄이는 선택 구조 적용

9. 배정 결과 저장 및 조회 API 구현

시간표 생성 결과를 DB에 저장하고 학과 코드 기준으로 조회할 수 있는 API 구현

```
def make_assign_table(request):
    code = request.GET.get('facultyCode', None)

    assign_table()

    filtered_data = AssignTable.objects.filter(
        FacultyCode=code
    )

    return JsonResponse(
        list(filtered_data.values()),
        safe=False
    )
```

```
def get_assign_table(request):
    code = request.GET.get('facultyCode', None)

    if code is None:
        return JsonResponse("")

    filtered_data = AssignTable.objects.filter(
        FacultyCode=code
    )

    return JsonResponse(
        list(filtered_data.values()),
        safe=False
    )
```

API 경로

```
urlpatterns = [
    path("table/data/course/import/", views.make_course),
    path("table/data/resource/import/", views.make_resource),
    path("table/assign/create/", views.make_assign_table),
    path("table/make/", views.make),
    path("table/view/", views.get_assign_table),
    path("courseoffer/view/", views.get_father_course_offer),
]
```

코드 설명

- 단일 시간표를 바로 확정하지 않고 여러 후보를 생성한 뒤 비교
- 제약 위반 수와 미사용 강의실 수를 기준으로 후보 평가
- 최종 결과의 충돌 가능성을 줄이는 선택 구조 적용